

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

TRAINING TR-102 REPORT DAY 22

22 JULY 2025

Overview:

Today I implemented a real-time face recognition system using Python. Unlike basic face detection, which only finds where a face is located, face recognition helps identify who the person is by comparing the live face with stored known images. This project focused on building an automatic attendance system using a webcam, where recognized individuals are marked as "present" in a CSV file. I used the powerful `face_recognition` and `OpenCV` libraries to handle both the facial processing and visual output.

Learning Objectives:

- Understand how face recognition differs from face detection.
- Learn to encode known faces using the `face_recognition` library.
- Perform real-time face matching using webcam feed.
- Automate attendance tracking and save it to a CSV file.

Main Concepts:

Today I implemented a face recognition system, which is more advanced than just face detection. Instead of only finding where faces are, I learned how to identify who the person is by comparing their face with already known images.

I used a powerful library called `face_recognition`, which makes it easy to:

- Encode known faces (like students or employees).
- Compare live faces from the webcam.
- Draw boxes and labels on recognized faces.
- Save their name and time in an attendance file.

Code Features:

Here's what my code did step-by-step:

1. Loaded images of known people (from the dataset/ folder) and converted them to face encodings, which are like unique digital signatures of each face.
2. Opened the webcam and captured frames in real time.
3. Resized and converted each frame to RGB (as required by the face_recognition library).
4. Detected all faces in the current frame and generated encodings for them.
5. Compared each new face with the known encodings using:
 - a. `compare_faces()` → checks if it matches.
 - b. `face_distance()` → finds how close it is (the smaller, the better).
6. If a match was found:
 - a. Displayed the name of the person on the webcam.
 - b. Drew a green rectangle around the face.
 - c. Marked attendance by saving their name and time in a CSV file.
7. If no match was found:
 - a. Displayed "Unknown".
 - b. Drew a red rectangle around the face.

Key Highlights:

- Built a full-face recognition-based attendance system using Python.
- Learned to use the face_recognition library to encode faces and identify people.
- Used OpenCV for webcam access and drawing on frames.
- Understood the concept of face encodings, comparison, and distance metrics.
- Practiced file handling with CSV to store attendance logs.
- Made the system efficient by resizing frames before processing (to make it faster).

Conclusion:

Today's session was highly productive and practical. I successfully created a working model of a face recognition attendance system. I used the modern and accurate `face_recognition` library for face encoding and matching. The system worked in real-time, detected both known and unknown faces, and saved recognized individuals with timestamps into a CSV file. This experience helped strengthen my understanding of real-world AI applications and improved my confidence in using OpenCV and machine learning libraries in Python.